

Explaining the Spaces Architecture

A speaking guide. Read this, not the technical document, before you present.

The idea in one sentence

Spaces has three parts - the app people use, the rules that sit behind it, and the records it keeps - and the middle part is where every decision gets made, which is what makes the approvals and the attendance trustworthy.

If you only get one sentence out, that is the one. Everything below is support for it.

The analogy to lead with

Treat it like walking into an office building.

- The **app** is the lobby. It is what you see, and it is where you say what you want.
- The **server** is the security desk. It checks your ID is genuine, looks up who you are today, checks whether your role is allowed in that room, and checks the room is actually yours.
- The **database** is the records room. Nobody walks in. The security desk goes and gets what you asked for.

The point of the analogy is the second part. A lobby can be decorated any way you like - it is the security desk that makes the building safe. Same here: the checks live on our server, not in the browser, because anything running on someone's own device can be tampered with.

The 60-second version

Deliver this without the diagram. It should sound like talking, not reading.

"Spaces is an events platform for our own employees. Somebody opens it, browses what is happening, and joins. Organizers run events, and an admin approves them before anyone can see them."

"Under that it is three layers. There is the app itself, on your phone or your laptop. Behind it is our server, which is where every rule lives - is this person signed in, what is their role, are they allowed to do this, is this actually their event. And behind that is the database, which is the one place the real records live."

"The important part is the middle. All the checking happens on our server, never on the person's device. That is why we can trust that an event really was approved by an admin, and that an attendance record is real."

The 3-minute walkthrough

Now put the diagram up. Walk it top to bottom, one numbered box at a time, and say what each stop is for before you say what it is made of.

Point at	Say roughly this
1. The app people use	"This is the part people actually touch. Three kinds of people use it - employees, organizers and admins - and each one sees a different set of screens. When you sign in, the app holds on to a pass, and it shows that pass every single time it asks the server for anything."
The arrow down	"So every action - joining an event, creating one, approving one - goes down this arrow, and it always carries the pass."
2. The rules	"This is our server, and this is the heart of it. Four checks, always in this order. Is the pass real. Who are you today - and we look that up fresh every time, so if an admin changes somebody's role it takes effect immediately, not tomorrow. Is your role allowed to do this. And finally, is this actually yours - an organizer can edit their own event, not someone else's."
The amber strip	"All four of those happen here, on our server. Never in the browser. Anything running on a person's own device can be changed by them, so it cannot be the thing that decides what they are allowed to do."
3. The records	"And this is the database - one copy of what is true. People, events, sign-ups, rewards. It also does the clever bits: finding events near you for the map, and adding up the rewards for the leaderboard. Notice nothing from the top reaches it directly. Everything goes through the middle."

Say it this way, not that way

Every term on the left will lose part of the room. The right-hand column says the same thing and keeps everyone.

Instead of	Say
Frontend / React SPA	the app people use, what you see on screen
Backend / Express API	our server, where the rules live
Database / Postgres	the records, the one place the real data lives
JWT / token	a sign-in pass the app carries
Authentication	checking the pass is real
Authorisation / role check	checking what that person is allowed to do
Endpoint / API route	a request the app can make
Query	looking something up
PostGIS / geospatial	working out which events are near you
Hashing / bcrypt	passwords are stored scrambled, so even we cannot read them
Approval workflow	an admin has to say yes before anyone sees it

The three points worth landing

If the room remembers nothing else, aim for these. Each one is a decision, with a reason - that is what makes it sound considered rather than accidental.

- **The rules live in one place.** Not scattered across screens. One server, one set of checks, every request. That is why a rule cannot be skipped by going a different way round.
- **We check who you are every single time.** We do not take your word for it from when you signed in. If your role changes, it applies straight away.
- **Nothing reaches the records directly.** The app never touches the database. It asks, and the server decides whether to go and get it.

Questions you will actually get

"Is this secure?"

"Passwords are stored scrambled, so nobody can read them back - not even us. You get a sign-in pass that expires. And every request is re-checked on the server, so the app on your phone cannot grant itself permissions it does not have."

"What happens if two people join the last slot at the same time?"

Be straight: "Right now the number of places is shown but not enforced at the moment of joining, so that race is possible. It is on the list, and the fix is to make the database enforce it rather than the app." Saying this before you are caught on it reads as competence. Bluffing does not.

"Can an organizer edit someone else's event?"

"No. That is the fourth check. The server confirms the event actually belongs to them before it saves anything."

"How does the map know what is near me?"

"The database works that out. We ask it for events within a certain distance and it returns them already sorted by how close they are - so your phone is not downloading everything and sifting through it."

"Why build it rather than buy something off the shelf?"

"Because the approval flow is ours. An event needs an admin to sign it off, and an employee needs approving before they can organise. A generic events tool does not have that, and it is the part that makes this usable inside the company."

"How hard is it to add a feature?"

"The three layers are separate, so most changes touch one of them. A new screen does not require touching the records. A new rule does not require touching the screens."

"What would you do next?"

"Enforce the place limit properly, with a waiting list - the records are already set up to hold one. Then give out the rewards automatically when attendance is marked, so joining an event runs through to the leaderboard without anyone doing it by hand."

If someone technical is in the room

Do not switch registers for the whole talk - one person's curiosity is not worth losing everyone else. Give a one-line answer and offer the detail afterwards.

If they ask	One line back
What is the stack?	"React and Vite on the front, Express on Node behind it, Supabase Postgres underneath. Happy to go through the detail after."
Why not call the database directly?	"We wanted one place where the rules are enforced, and we did not want a database credential sitting in the browser."
Is Row Level Security on?	"Not currently - the server holds the only credential and enforces everything. If anything ever talks to the database directly, RLS becomes mandatory. It is written up as a known gap."
Where is the token stored?	"Local storage today. Moving it to a cookie is the hardening step that is on the list."
Any tests?	"Not yet. The highest value ones would be tests that drive the API end to end, and that is what I would write first."

Then point them at the technical document. It has all of it, including the limitations, written down.

Presenting: a few rules

- **Say what it is for before what it is built with.** "This is where the rules live" lands; "this is the Express layer" does not.
- **Follow the arrows.** The diagram is one journey, top to bottom. Tell it in that order and nobody gets lost.
- **Name the limitations yourself.** You already know the two: places are not enforced at the moment of joining, and rewards are not yet given out automatically. Saying them first is strength.
- **Do not read the boxes aloud.** They can read. Say the thing the box does not say - why it is there.
- **Keep the technical document closed** unless someone asks for that level. Offer it; do not open it.